

AI Prompt Integration Guide (Sprint 5)

Coordinación Multi-IA con Metodología ASPECCT

Estado Actual del Proyecto

Objetivo: Asegurar compatibilidad total entre el backend Laravel 12 (Passport 13.2) y el frontend React + TypeScript mediante prompts estructurados y coordinación de agentes IA.

Fase	Estado	Responsable
1 Planificación Arquitectura	✓	Claude 3.5
2 Diseño UI/UX	✓	ChatGPT-4o
3 Implementación Base Frontend	✓	React Frontend Assistant + Copilot
4 Ajuste y Documentación	✓	Laravel GPT + Agente de Código
5 Compatibilidad API	🟡 En revisión	Laravel GPT
6 Depuración y QA	🟡 Pendiente	Claude + ChatGPT-4o

Verificación de Compatibilidad Backend/Frontend

Modelo: Laravel GPT

Archivos necesarios: - AUTHENTICATION_IMPLEMENTATION.md
- BACKEND_INTEGRATION_CHECKLIST.md
- Swagger.md

Prompt ASPECCT:

A: Actúa como especialista backend en Laravel 12.0 y APIs REST.
S: Fase de verificación de compatibilidad entre frontend y backend.
P: Validar que los servicios Axios generados en React coincidan con los endpoints del Swagger y la configuración Passport.
E: Devuelve lista de endpoints compatibles, diferencias detectadas y sugerencias de ajuste.
C: No inventes endpoints; usa solo los definidos en Swagger.
C2: Verifica headers, tokens, roles y expiración de refresh tokens.
T: Técnico, preciso y orientado a integración real.

Resultado esperado: Informe de compatibilidad + checklist de ajustes mínimos.

Revisión de Tipado y Coherencia Semántica

Modelo: Claude 3.5

Archivos necesarios: - Carpeta `/src/` del frontend
- AUTHENTICATION_IMPLEMENTATION.md

Prompt ASPECT:

A: Actúa como revisor de código TypeScript y arquitecto de frontends.
S: Fase de depuración semántica tras integración del backend.
P: Detectar incoherencias en tipos, props o datos entre componentes y API.
E: Lista de inconsistencias con ejemplos corregidos.
C: No reescribas todo el código; sugiere correcciones localizadas.
C2: Asegúrate de que los tipos TS coincidan con los esquemas JSON de la API.
T: Técnico, analítico y conciso.

Resultado esperado: Informe de inconsistencias semánticas + fragmentos corregidos.

Test de Integración y QA Final

Modelo: ChatGPT-4o (modo Agent)

Archivos necesarios: - SPRINT_5_COMPLETION_SUMMARY.md
- BACKEND_INTEGRATION_CHECKLIST.md
- Carpeta `/src/`

Prompt ASPECT:

A: Actúa como auditor QA full-stack.
S: Fase final de validación antes del cierre del sprint.
P: Verificar la coherencia funcional, compatibilidad y experiencia UX/UI.
E: Checklist de validaciones y recomendaciones finales de mejora.
C: No generes nuevo código, solo evalúa lo existente.
C2: Usa la documentación del Sprint 5 y el checklist de integración como referencia.
T: Formal, técnico y evaluativo.

Resultado esperado: Informe QA con validaciones de rendimiento, UX/UI y compatibilidad.

Desarrollo Continuo (día a día)

Modelo: Copilot + React Frontend Assistant

Archivos necesarios: Ninguno (funciona dentro del IDE)

Comentarios tipo:

```
// TODO: Implement refresh token renewal with axios interceptor using  
AUTHENTICATION_IMPLEMENTATION.md rules  
// TODO: Add XP Level progress bar component based on XP_LEVELS_ROADMAP.md
```

Resultado esperado: Código complementado automáticamente y adaptado al contexto actual del proyecto.

Resumen Final

-  El punto 4 de la hoja de ruta está completamente finalizado.
- Los prompts restantes mantienen compatibilidad, validan tipos y aseguran QA final.
- La documentación técnica y los endpoints del backend están sincronizados.

 *Versión Sprint 5 – AI Integration & Compatibility Workflow.*